

Catverdose

Java · Spring Boot | GitHub github.com/Catverdose

동시 수정, 비동기 처리, 실시간 스트리밍처럼

실패했을 때 데이터와 시스템 상태가 어긋날 수 있는 경계를 파고듭니다.

기능이 동작하는 데서 끝내지 않고, 재현 가능한 테스트와 측정으로 정합성을 확인합니다.

FOCUS

Data Consistency

transaction boundary · DB constraint · concurrency control

Failure Handling

async processing · scheduler · SSE · recovery

Verification

integration test · regression test · benchmark

Backend Infrastructure

Docker · Testcontainers · CI/CD · observability

PROJECTS

프로젝트	구분 · 내 역할	핵심	page
PetCoupon	팀 (백엔드 6명) 이벤트·쿠폰 관리, 관리자 운영·모니터링	관리자 수정 ↔ 발급·스케줄러 경합 제어, SSE 모니터링의 오류 자기증폭 차단	2
Engineering Memory	개인 설계·구현·문서 전체	비동기 색인의 stale write 차단, LLM 장애 시 대화 상태 복구, DB 수준 데이터 격리	4
Coupon Concurrency	개인 팀 실험을 확장, 설계·측정 전체	동시성 제어 9개 전략 비교. 초과 발급 없이도 깨지는 지점(정상 회원 탈락, 연결 고갈, lost update) 확인	6
Vector DB Benchmark	UBot 팀 내 실험 benchmark harness 구현	5개 DB · 14개 구성 · 620 measurements, 단일 수치로 결정하지 않는 비교 설계	7
UBot Backend	팀 개발·테스트 실행 기반	pgvector·PostGIS 의존 서비스의 로컬·CI 환경 일치, Testcontainers 테스트 격리	8
Planly	팀 → 개인 확장	Todo-공유 Calendar 연동, JWT 인증·소유권 검증	8

TECH

주력 Java 21 · Spring Boot · JPA · PostgreSQL · MySQL

사용 경험 Redis · pgvector · PostGIS · Flyway · Docker · Testcontainers · GitHub Actions · SSE · Nginx · Go · Ollama · k6

PetCoupon 반려동물 이벤트 기반 선착순 쿠폰 발급 시스템

팀 프로젝트 · 백엔드 6명

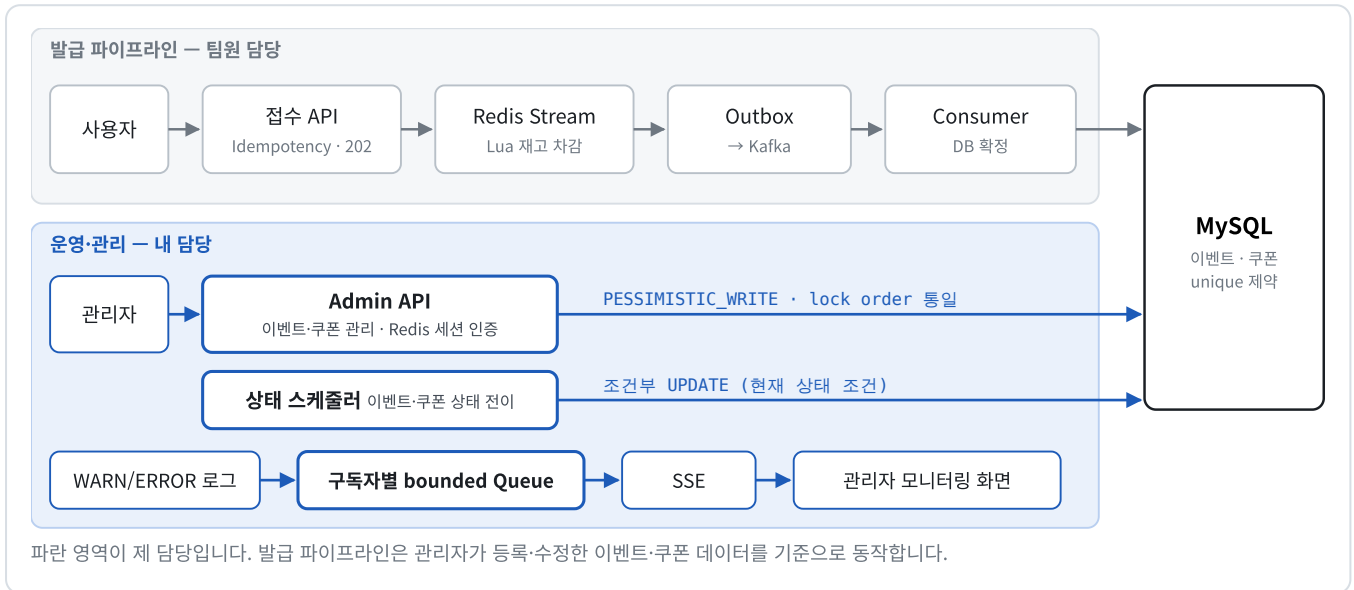
담당: 이벤트·쿠폰 관리, 관리자 운영·모니터링

Merged PR 24개

통합 테스트 시나리오 16개 (TC-01~06, 20~29)

Java 21 · Spring Boot 4.1 · Spring Data JPA · MySQL 8.0 · Redis 7.2 · Kafka 3.7 · JUnit 5 · Testcontainers · Awaitility · k6

대량 동시 요청에서도 초과 발급 없이, 사용자당 1장만 발급하는 것이 목표인 시스템입니다. 발급 요청은 Redis·Kafka 파이프라인이 처리하고, 저는 그 파이프라인이 기대는 이벤트·쿠폰 데이터를 관리하고 운영자가 시스템 상태를 실시간으로 보는 영역을 맡았습니다. 코드 영역은 `event/`, 관리자 컨트롤러, `monitoring/` 입니다.



관리자 SSE 모니터링의 오류 자기증폭 차단

- 상황** 운영자는 WARN/ERROR 로그를 SSE로 실시간 구독합니다. 그런데 모니터링 화면의 연결이 끊기면 오류가 스스로 불어나는 현상이 있었습니다.
- 원인** `client disconnect` → 예외 → `exception handler`가 로깅 → 그 로그가 SSE로 다시 전송 → 전송 실패 → 다시 로깅 모니터링 대상인 로그가 모니터링 채널을 통해 다시 오류를 만드는 feedback loop였습니다.
- 결정**
- loop를 추적해 끊고, 구독자마다 별도 Queue를 뒀 느린 client가 다른 구독자와 로그를 쓰는 요청에 영향을 주지 않게 격리
 - Queue가 가득 차면 **비즈니스 요청을 막지 않고 모니터링 이벤트를 버림**. 대신 client에 `events-dropped`를 알리고 `monitoring.sse.events.dropped` 지표로 손실량을 드러냄
 - 로그 마스킹 비용에 상한을 뒀 모니터링이 서비스 비용을 키우지 않게 함
- 판단** 모니터링의 완전성보다 서비스 가용성을 우선했습니다. 다만 무엇을 잃었는지는 숨기지 않고 지표로 남겼습니다.

PetCoupon 데이터 경합 · 조회 · 인증

관리자 수정 ↔ 발급·스케줄러 동시 갱신 경합

- 상황** 관리자의 쿠폰 수정, 발급 처리, 상태 전이 스케줄러가 같은 이벤트·쿠폰 데이터를 동시에 갱신합니다.
- 위험** 나중에 쓴 쪽이 앞선 변경을 덮어쓰거나, 경로마다 락을 잡는 순서가 다르면 서로를 기다리며 교착에 빠질 수 있습니다.
- 결정**
- 관리자 수정 경로는 `PESSIMISTIC_WRITE` 로 직렬화하고, 모든 경로에서 락 획득 순서를 통일
 - 상태 전이는 현재 상태를 조건에 둔 조건부 UPDATE로 구현해, 같은 전이가 두 번 적용되지 않게 함

목록 조회와 실시간 조회의 경로 분리

- 결정** 목록 조회와 실시간 조회의 데이터 소스를 나누고, 실제 실행되는 SQL 수를 테스트로 고정해 조회 로직이 바뀌어도 쿼리 수가 슬그머니 늘지 않게 했습니다.

관리자 인증

- 결정**
- Redis 기반 관리자 세션. 8시간 만료와 폐기 구조를 둠
 - 401 응답은 토큰 누락·무효·만료를 구분하지 않음. 공격자에게 단서를 주지 않기 위함
 - 관리자 인증 헤더(`X-ADMIN-KEY`)가 필요한데 브라우저 기본 `EventSource` 는 임의 헤더를 붙일 수 없어 fetch 기반 SSE client로 연결

팀 전체 검증 결과 — 발급 파이프라인 포함, 팀 공동 성과

20,000 요청 부하 테스트 · 초과 발급 / 중복 발급 0건 · 1,030 TPS

남은 과제 (운영 영역)

- Redis Stream DLQ는 조회·재처리용 관리자 API가 아직 없어 `XRANGE` 로 수동 확인만 가능합니다. 처리 절차를 먼저 정한 뒤 관리자 기능으로 만들어야 합니다.

Engineering Memory

개인 개발 기록 기반 RAG 지식 어시스턴트

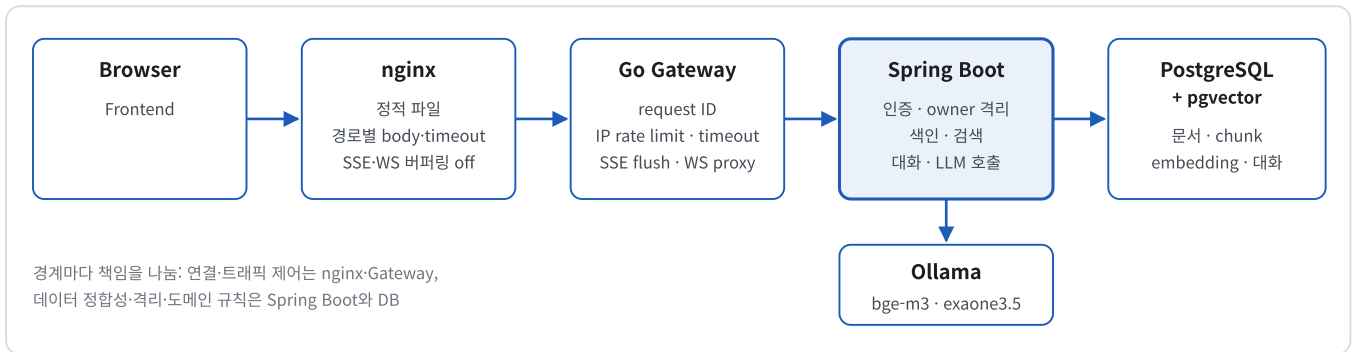
개인 프로젝트 · 설계·구현·문서 전체

Phase 1 완료 (2026-09)

MIT License

Java 21 · Spring Boot 4.1.1 · Go 1.22+ · PostgreSQL 17 + pgvector · nginx 1.27 · Ollama (bge-m3 1024d, exaone3.5:7.8b) · Docker Compose

프로젝트 문서, 장애 기록, 기술 결정, 학습 노트를 저장하고 **그 기록을 근거로만 답하는** 어시스턴트입니다. 근거 문서가 없으면 답을 지어내지 않고 `NO_CONTEXT` 를 반환합니다. 계정은 관리자가 발급하며, 여러 사용자의 데이터가 한 DB에 섞여 있어도 서로 보이지 않아야 합니다.



비동기 색인 중 수정된 문서가 과거 결과로 덮이는 문제

- 상황** 등록 API는 색인을 기다리지 않고 `202 Accepted` 를 반환합니다. 색인은 문서 단위 큐(최대 동시 2개)에서 chunking → batch embedding 순으로 진행됩니다.
- 위험** 색인 도중 사용자가 문서를 수정하면, 늦게 끝난 이전 작업이 새 버전 문서를 과거 embedding으로 덮어쓸 수 있습니다.
- 결정**
- 저장 직전에 `version · status · attempt UUID` 가 시작 시점과 같은지 확인. 다르면 결과를 버리고 새 색인을 예약
 - 기존 chunk 삭제 · 새 chunk 삽입 · 상태 완료를 **한 트랜잭션**으로 처리해 중간 상태가 보이지 않게 함
 - `PENDING → READY / FAILED` 상태, bounded queue, 작업 coalescing, 재기동 복구 구성



Engineering Memory 대화 · 격리 · 설계 결정

LLM 처리 중 장애가 나도 대화 상태를 잃지 않기

상황 로컬 GPU에서 LLM이 답을 생성하는 구간은 길어서, 그 사이에 서버 재기동이나 장애가 끼어들 여지가 큼니다.

- 결정**
- beginTurn: 대화 행을 잠그고 사용자 메시지와 GENERATING 상태의 답변 메시지를 **먼저 저장** → 검색 → 프롬프트 조립 → 생성 → completeTurn: 답변과 근거 저장
 - 생성 도중 서버가 재기동되면 기동 시 남아 있는 GENERATING 을 FAILED 로 전환. 사용자는 영원히 멈춘 답변이 아니라 실패 상태를 봄
 - 유사도 0.45 이상인 근거가 없으면 NO_CONTEXT 를 저장하고 LLM 호출을 생략. 근거 없는 답변과 불필요한 GPU 사용을 함께 막음

데이터 격리를 애플리케이션과 DB 양쪽에서

- 결정**
- owner ID는 인증 principal에서만 꺼내고 요청 본문 값은 믿지 않음. 모든 조회에 owner 조건
 - chunk·메시지는 (id, owner_id) 복합 FK로 참조. 코드에 결함이 있어도 다른 사용자 데이터와 섞이는 행은 DB가 거부
 - 벡터 검색은 owner · READY · version · model 조건을 정렬·LIMIT 전에 적용. 무관한 행이 Top-K 자리를 차지하지 않게 함

DESIGN DECISIONS

결정	이유
소규모 데이터에서는 HNSW 대신 exact scan	근사 검색의 recall 손실보다 정확도를 우선. 데이터가 커지면 재검토
chunk마다 embedding_model 저장	모델이 다른면 검색에서 제외하고 needsReindex 표시. 호환되지 않는 벡터가 섞이지 않게 함
동시성 상한: 채팅 50, 채팅 중엔 색인 배치 30초 양보	GPU 한 장을 채팅과 색인이 나눠 쓰므로 대화 지연을 우선 보호
기동 시 프롬프트 예산 검증	$\text{max-prompt-chars} \times 3 + \text{num-predict} + 64 \leq \text{num-ctx}$. UTF-8 최악의 경우에도 context window를 넘지 않게 보장
owner 내 동일 SHA-256 문서 거부	같은 근거가 검색 결과를 중복으로 차지하지 않게 함

남은 과제

- 마이그레이션 도구 미도입: ddl-auto: none , schema.sql 은 새 볼륨에서만 적용. 스키마를 바꾸려면 볼륨을 새로 만들어야 함
- 실패한 색인 작업은 자동 재시도하지 않음. 현재는 수동 재색인으로만 복구

검증

./gradlew test · ./gradlew integrationTest · go test ./... · 기동 후 /readyz health check. 설계·API·설정·장애 대응은 docs/에 문서화했습니다.

Coupon Concurrency Experiment

초과 발급 없이도 깨지는 지점

개인 실험 · 설계·구현·측정·분석 전체

PetCoupon 팀 실험을 개인적으로 확장

9개 전략 · 시나리오 3종

Java 21 · Spring Boot 4.1.0 · Spring Data JPA · MySQL 8.4 · Redis 7 · k6 2.2.0 · 로컬 단일 호스트 1차 실험 (2026-08)

같은 발급 흐름에서 **재고 예약 전략만 바꿔** 9개 전략(기준선 · JVM Lock · DB 3종 · Redis 4종)을 비교했습니다. 각 회원이 한 번씩 요청하는 시나리오에서는 정합성 문제가 락이 없는 **DIRECT**에서만 보였습니다. 그래서 **한 회원이 서로 다른 requestId로 3번씩 요청**하도록 바꿨습니다(재고 10,000 · 회원 10,000명 · 30,000건). 1인 1매 제약은 모든 전략이 지켰지만, 나머지 조건까지 통과한 전략은 단일 인스턴스의 **JVM_LOCK** 뿐이었습니다.

전략 (VU 50)	대상자 전원 발급	응답 분류	재고 원장	통과
JVM_LOCK	10,000	일치	일치	통과
PESSIMISTIC	10,000	중복 2건이 SOLD_OUT	일치	✗
CONDITIONAL	10,000	중복 2건이 SOLD_OUT	일치	✗
DIRECT	10,000	일치	차감 2,502 / 발급 10,000	✗
REDIS_DECR	9,999	일시 SOLD_OUT 13건	일치	✗
REDIS_LUA	9,998	일시 SOLD_OUT 13건	일치	✗
REDIS_LOCK	9,999	한도 초과 287건	일치	✗
OPTIMISTIC	9,997	한도 초과 11건	일치	✗
REDIS_WATCH	946	무응답 26,991건	일치	✗

실행 순서를 뒤집어 한 번 더 돌렸을 때도 통과한 전략과 실패 유형은 같았습니다. 조건별 1회, 단일 호스트 결과라서 성능 순위는 확정하지 않았습니다.

Redis 선예약이 정상 회원을 탈락시킴

원인 중복 요청이 회원 중복 판정 전에 Redis 재고를 먼저 잡습니다. DB 유니크 충돌로 보상하기 전까지 재고가 비어 보여서, 그사이 다른 회원이 품절 응답을 받았습니다. 최종 원장은 맞았습니다.

다음 회원 중복 검사와 재고 예약을 Lua script 한 번에 묶는 변형으로 비교할 예정입니다.

REDIS_WATCH의 연결 고갈

관찰 요청의 약 90%가 응답을 받지 못했습니다. TIME_WAIT 약 14,900개가 로컬 동적 포트 16,384개를 거의 채워 **Address already in use**가 발생했습니다. VU 10·50·100·200에서 모두 재현됐습니다.

원인 WATCH 충돌 재시도마다 Redis 연결을 새로 만들고 닫는 구조였습니다. 다른 전략을 오염시키지 않도록 Watch만 단독 실행해 원인을 분리했습니다.

그 밖의 관찰

- **DIRECT lost update**: 응답은 모두 맞았지만, 발급 10,000건 중 재고 차감은 2,502건만 반영
- **SOLD_OUT 오분류**: 회원 중복 사전 조치가 최초 발급 커밋 전에 통과한 뒤 재고 0을 만남 (check-then-act)

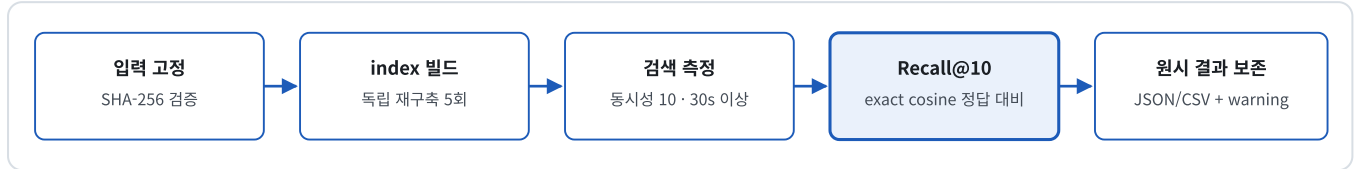
Vector DB Benchmark RAG 서비스의 Vector DB 후보 비교 harness

UBot 팀 프로젝트 내 실험

benchmark harness 구현

fairness-v2 재측정 (2026-09)

질문은 "어느 DB가 가장 빠른가"가 아니라 "같은 검색 품질과 같은 자원 조건에서 비교하면 무엇이 다른가"였습니다. 품질이 다른 두 구성을 latency만으로 비교하면 결론이 왜곡되기 때문입니다.



실험 조건

대상 (14개 구성)	pgvector (HNSW, IVFFlat) · Qdrant (HNSW) · Weaviate (HNSW, HFresh) · Milvus (HNSW, IVF_FLAT, IVF_SQ8, IVF_PQ, DISKANN) · OpenSearch (Lucene / Faiss / JVector)
데이터	합성 10k chunk · BGE-M3 dense 1024d · cosine · Top-10
자원	DB당 4 vCPU / 8 GiB 고정 (Milvus는 etcd-MinIO 포함)
측정 규모	검색 설정 124개 × 독립 재구축 5회 = 620 measurements · 측정점마다 자원 샘플 30개 이상
지표	Recall@10 · latency · QPS · CPU · RAM · index readiness

신뢰성을 위한 설계

같은 입력, 같은 정답

모든 DB에 같은 입력이 들어갔음을 SHA-256으로 고정하고, Java exact cosine search 결과를 Ground Truth로 삼았습니다. calibration query와 evaluation query는 분리했습니다.

불리한 데이터도 지우지 않음

측정 이상치와 warm-up 미달을 제거하지 않고 원시 결과와 warning으로 남겼습니다 (warning 170건). Milvus DISKANN은 5회 관찰에서 Recall 변동이 확인되었습니다.

결론: 탐색 결과만으로 제품을 확정하지 않음

- 이번 결과는 합성 10k · Top-10 · 동시성 10 조건의 **탐색 자료**입니다. Recall이 다른 구성을 속도만으로 줄 세우지 않았습니다.
- 코드에 남아 있던 임계값(Recall 0.95, p95 30ms, RAM 2GiB)은 서비스 요구사항이라는 근거가 없어 탈락 판정에 쓰지 않았습니다.
- 다음 단계를 정의했습니다: 실제 1k~10k chunk 측정 → pgvector baseline 비교 → 독립 holdout 검증 → 운영 복잡도 검토.

UBot Backend

통신사 고객 상담용 RAG 챗봇

팀 프로젝트

담당: 팀 개발·테스트 실행 기반 (인프라)

Java 21 (Gradle toolchain) · Spring Boot 4.1.1 · Spring Security + JWT · PostgreSQL 18 + pgvector 0.8.6 + PostGIS 3.6.4 · Flyway (V1-V11) · Spring AI 2.0.1 · Ollama bge-m3 · Testcontainers · Docker · GitHub Actions

질문을 embedding해 FAQ를 벡터 검색하고 LLM이 답하는 RAG 흐름에, 매장 근접 검색(PostGIS)까지 쓰는 서비스입니다. 핵심 기능이 PostgreSQL 확장에 의존하기 때문에, **팀원 누구나 같은 환경에서 실제 기능을 개발하고 테스트할 수 있는 기반**이 먼저 필요했습니다. 이 확장 기능들은 다른 DB로 흉내 낼 수 없습니다.



로컬과 CI의 환경 차이 제거

- 결정**
- 로컬 Compose와 CI가 **같은 PostgreSQL Dockerfile**을 사용
 - pgvector / PostGIS extension 생성 책임을 Flyway 한 곳으로 통일. 환경마다 생성 주체가 달라 생기는 차이를 없앴

테스트 격리와 실제 기능 검증

- 결정**
- Testcontainers로 테스트마다 독립 DB를 생성. 개발 DB와 테스트 환경이 서로 오염되지 않음
 - 대체 구현 대신 실제 vector 저장·검색과 PostGIS spatial function을 CI에서 실행해, 확장 기능에 의존하는 코드가 깨지면 CI에서 바로 드러나게 함

배포 이미지

- 결정** Java 21 multi-stage Docker image로 빌드 도구를 실행 이미지에서 분리하고, 배포용 Compose를 따로 구성했습니다.

OTHER WORK

Planly Todo와 공유 Calendar를 연결한 일정 관리 서비스

3인 팀 미니 프로젝트를 개인 저장소로 이어받아 재구성

102 commits

Java 21 · Spring Boot 4.1 · Spring Security · JWT (JJWT 0.12) · Spring Data JPA · MySQL · React 19 · Vite

영역	원래 팀 프로젝트	개인 확장
구조	단일 Spring Boot 프로젝트	backend / frontend 분리, React 19 SPA
인증·권한	JWT 인증	리소스 소유권 검증
Todo	기본 CRUD · 완료 처리	검색·필터·페이지네이션, 하위 Todo 계층, Eisenhower Matrix 분류
Calendar	기본 공유	Todo-Schedule 연동, 이메일 기반 멤버 관리